

— IE437 · DATA-DRIVEN DECISION MAKING AND CONTROL — LECTURE 3

Bayesian Networks

A belief about many things is a graph

Jinkyoo Park · KAIST

Structure tames the joint — and once a decision hangs off the graph, every later lecture is already in the room

— HANDOFF

The handoff — belief over a whole system

Lecture 2 carried a belief over *one* parameter. Real systems have many, and they interact.

Where we are — one unknown becomes many

STAGES	static ● — ○	MODEL	data-driven ○ — ●	AGENTS	single agent ● — ○
--------	--------------	-------	-------------------	--------	--------------------

MODEL-BASED		DATA-DRIVEN
Static, single	optimisation (<i>Lec 1</i>)	Bayesian statistics (<i>Lec 2</i>) → Bayesian networks (<i>Lec 3</i>)

Lecture 2 held a distribution over a coin's bias, a rate, a weight vector. But a satellite does not fail one parameter at a time: a [battery failure raises the chance of an electrical failure](#), which raises the chance of a trajectory deviation and of a communication loss.

This chapter does not cross an axis. It stays in the same cell of the cube — static, data-driven, one agent — and [deepens it](#): belief over a *system* rather than a parameter, and then, in Act 4, belief with an action attached.

The wall — a joint distribution is exponentially large

For n binary variables the joint $p(x_1, \dots, x_n)$ is a table of 2^n rows, so specifying it takes

$$2^n - 1 \text{ independent numbers} \quad (M^n - 1 \text{ if each variable has } M \text{ values})$$

Five binary variables need 31 numbers. Twenty need 1 048 575. Thirty need over a billion. **Too many to store, far too many to estimate from data, and hopeless to sum over.**

The escape is structure: **a belief about many things is a graph.**

The thesis — two claims, and the second is the quiet one

CLAIM ONE

Acts 1–3

Conditional independence, drawn as a directed graph, turns one exponential table into [a product of small local tables](#) — and makes belief over a system storable, learnable, and computable.

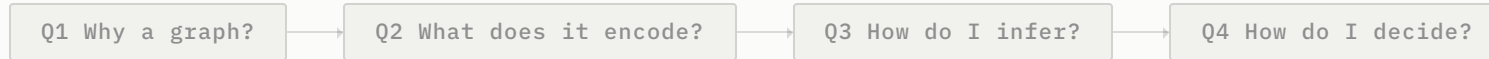
CLAIM TWO

Act 4

Add a **decision node** and a **utility node** to that graph and you have belief, action, and value in one object for the first time in this course. That object — the [decision network](#) — is a single-shot decision under uncertainty. Unroll it through time and it is the Markov Decision Process of Lecture 7.

So far the course has *modelled* (Ch 1–2). Here modelling and deciding are joined, in miniature, before time enters the picture. Lecture 7 will reach back to this slide by name.

The roadmap — four questions



- **Q1 — Why represent a joint as a graph?** The factorisation $p(x) = \prod_i p(x_i \mid \text{pa}_i)$, and the collapse in parameter count it buys.
- **Q2 — What does the graph actually encode?** **Conditional independence** — read off missing edges, and subtler than it looks.
- **Q3 — How do we answer questions with it?** **Inference**: enumeration, variable elimination, sampling; and the same machinery run through time.
- **Q4 — How do we add *decisions*?** **Decision networks**, maximum expected utility, and the seed of the MDP.

— ACT 1

The joint as a graph

Q1. Every distribution factorises somehow. A Bayesian network is a factorisation you can draw — and drawing it is what makes it cheap.

A joint, written out in full

Q1 Why a graph?

Q2 What does it encode?

Q3 How do I infer?

Q4 How do I decide?

A	B	C	$P(A, B, C)$
0	0	0	0.08
0	0	1	0.15
0	1	0	0.05
0	1	1	0.10
1	0	0	0.14
1	0	1	0.18
1	1	0	0.19
1	1	1	0.11

THREE BINARY VARIABLES, AND WHAT THE TABLE COSTS

$2^3 = 8$ rows — every combination A , B and C can take. The last is forced by the others, so $2^3 - 1 = 7$ free numbers.

The table is *complete*: every question about A , B and C is a sum over some of its rows, and nothing is assumed.

That completeness is exactly the problem. It is **useless at scale**, because a flat table has no way to say that A is irrelevant to C once B is known.

A graph can say it. That is the whole content of this act.

A directed acyclic graph, and the words for reading one

A **graph** is nodes and edges; a **directed** graph puts an arrow on each edge; a **directed acyclic graph (DAG)** is one in which no path following the arrows ever returns to where it started.

THE VOCABULARY

For an edge $x_i \rightarrow x_j$, x_i is a **parent** of x_j and x_j a **child** of x_i . Following arrows forward from a node reaches its **descendants**; following them backward reaches its **ancestors**.

pa_x denotes the set of parents of x — the only piece of notation this lecture really needs.

WHY ACYCLIC

A cycle would make a variable its own ancestor, and the product $\prod_i p(x_i \mid \text{pa}_i)$ would no longer be a distribution — there would be no order in which to generate the variables.

Acyclicity is what guarantees a **topological order**: a sequence in which every node comes after its parents. Sampling, elimination and learning all walk that order.

A DAG can be stored as an edge list or an adjacency matrix, but neither is how you should think about it. Think of it as a *recipe*: draw each variable once its parents have been drawn.

The factorisation — one joint, written locally

A **Bayesian network** is a distribution of the form $p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid \text{pa}_{x_i})$, drawn as a DAG in which each node is a variable and each arrow runs from a parent to a child.

THE SATELLITE, IN FIVE VARIABLES

Two causes, B *battery failure* and S *solar panel failure*, meet at E *electrical system failure*, which in turn produces D *trajectory deviation* and C *communication loss*.

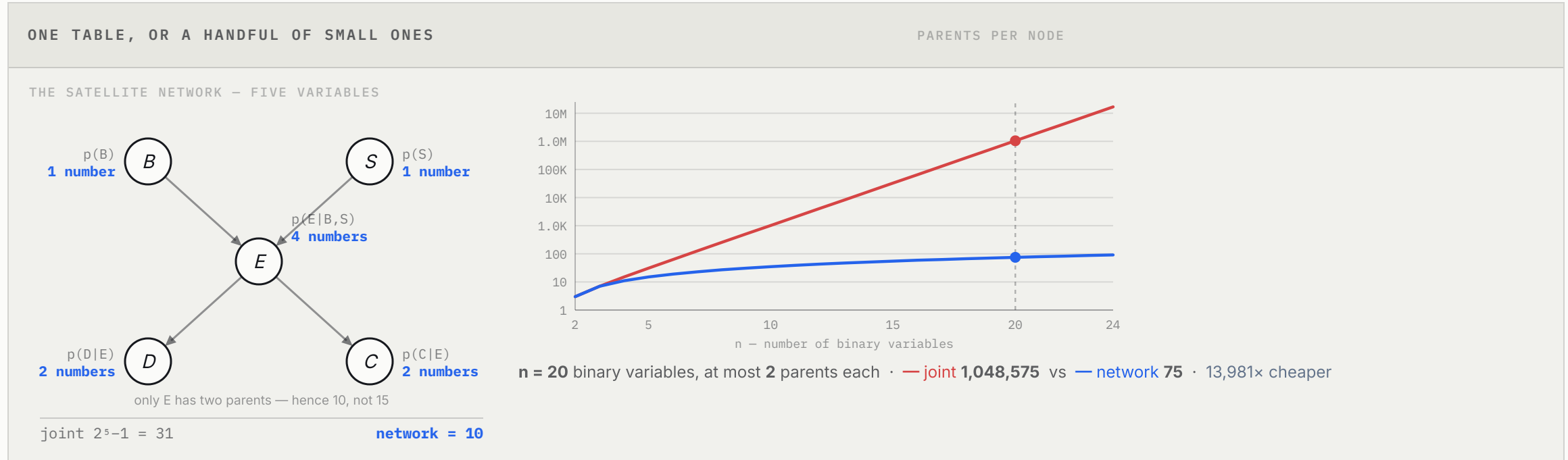
$$p(B, S, E, D, C) = p(B) p(S) p(E \mid B, S) p(D \mid E) p(C \mid E)$$

THE BILL

FACTOR	FREE NUMBERS
$p(B)$	1
$p(S)$	1
$p(E \mid B, S)$	4
$p(D \mid E)$	2
$p(C \mid E)$	2
total	10

against $2^5 - 1 = 31$ for the flat joint. **One giant table becomes five small ones.**

The collapse, in numbers



The joint's cost is $2^n - 1$ whatever you do. The network's cost is set by **how many parents a node has, not how many variables exist** — so it grows linearly while the joint explodes. Slide n and watch the gap open; the satellite sits at $n = 5$, where 31 becomes 10.

Every distribution is a Bayesian network — and that is the catch

The chain rule already writes any joint as a product of conditionals:

$$p(x_1, \dots, x_n) = p(x_n \mid x_1, \dots, x_{n-1}) p(x_{n-1} \mid x_1, \dots, x_{n-2}) \cdots p(x_1)$$

which is a Bayesian network — on the *fully connected* DAG. It costs exactly $2^n - 1$ numbers and buys nothing.

The saving is not in the factorisation. It is in **the edges you leave out**.

$p(E \mid B, S)$ rather than $p(E \mid B, S, D, C)$ is a claim about the world: once you know the battery and the panel, nothing else changes your belief about the electrical system. The DAG *is* the model, and every missing arrow is an assumption you are on the hook for. Act 2 says exactly which assumptions a given DAG makes.

— ACT 2

What the graph encodes

Q2. A missing edge is a conditional independence. Reading them off is easy in two cases out of three — and the third is where intuition fails.

Conditional independence — the meaning of a missing edge

Q1 Why a graph?

Q2 What does it encode?

Q3 How do I infer?

Q4 How do I decide?

INDEPENDENCE

$$X \perp Y \iff p(X, Y) = p(X)p(Y)$$

equivalently $p(X | Y) = p(X)$: learning Y tells you nothing about X .

CONDITIONAL INDEPENDENCE

$$X \perp Y | Z \iff p(X, Y | Z) = p(X | Z)p(Y | Z)$$

equivalently $p(X | Y, Z) = p(X | Z)$: **once you know Z , learning Y tells you nothing further about X .**

The DAG's basic promise is the **local Markov property**: a variable is independent of its non-descendants given its parents. In the satellite network, $C \perp B | E$ — knowing the battery failed does not move your belief about communication loss **if the electrical system's state is already known**. Likewise $D \perp S | E$.

That single sentence is what the missing edges $B \rightarrow C$ and $S \rightarrow D$ mean. It is also why $p(C | E)$ has two numbers instead of sixteen.

Three structures, three verdicts

Chain any two variables through a third and there are only three shapes. Whether X and Y are independent depends on the shape *and* on whether the middle node is observed.

STRUCTURE	SHAPE	$X \perp Y$ WITH Z UNOBSERVED	$X \perp Y$ GIVEN Z
chain	$X \rightarrow Z \rightarrow Y$	no — influence flows through	yes
fork (common cause)	$X \leftarrow Z \rightarrow Y$	no — a shared cause correlates them	yes
collider (common effect)	$X \rightarrow Z \leftarrow Y$	yes — separate causes	no

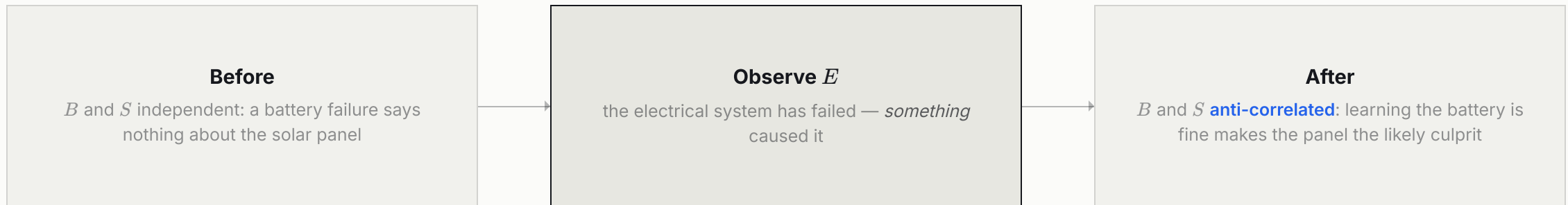
The first two say the same thing: observing the middle node **blocks** the path. The collider inverts it, and the algebra is one line each:

$$\text{fork: } p(x, y | z) = \frac{p(z)p(x | z)p(y | z)}{p(z)} = p(x | z)p(y | z) \quad \text{collider: } p(x, y | z) = \frac{p(x)p(y)p(z | x, y)}{p(z)} \neq p(x | z)p(y | z)$$

The offending term is $p(z | x, y)$: the effect couples its causes, and conditioning on it cannot factor that coupling away — whereas *marginalising* over z would. So the two operations do opposite things: on a fork, summing Z out **connects** X and Y while conditioning on it **separates** them; on a collider, exactly the reverse.

Explaining away — the collider, in words

nothing changed in the world; something changed in what you know



Each cause **explains away** the other. This is the one place where the graph predicts a dependence that the causal story does not obviously suggest — and it tells you in advance exactly when it will happen. It is also why careless conditioning manufactures correlations: **select a sample on a common effect and you couple its causes.**

Wet grass — explaining away, with numbers

Example 3.1

THE MODEL

S sprinkler on · R raining · T Tracey's grass wet · J Jack's grass wet

$$S \rightarrow T \leftarrow R \rightarrow J$$

$$p(T, J, R, S) = p(T \mid R, S) p(J \mid R) p(R) p(S)$$

$p(S=1) = 0.1$, $p(R=1) = 0.2$; the grass is certainly wet if it rained, wet with probability 0.9 if only the sprinkler ran, and dry otherwise; $p(J=1 \mid R=1) = 1$, $p(J=1 \mid R=0) = 0.2$.

THE TWO QUERIES

$$p(S=1) = 0.100$$

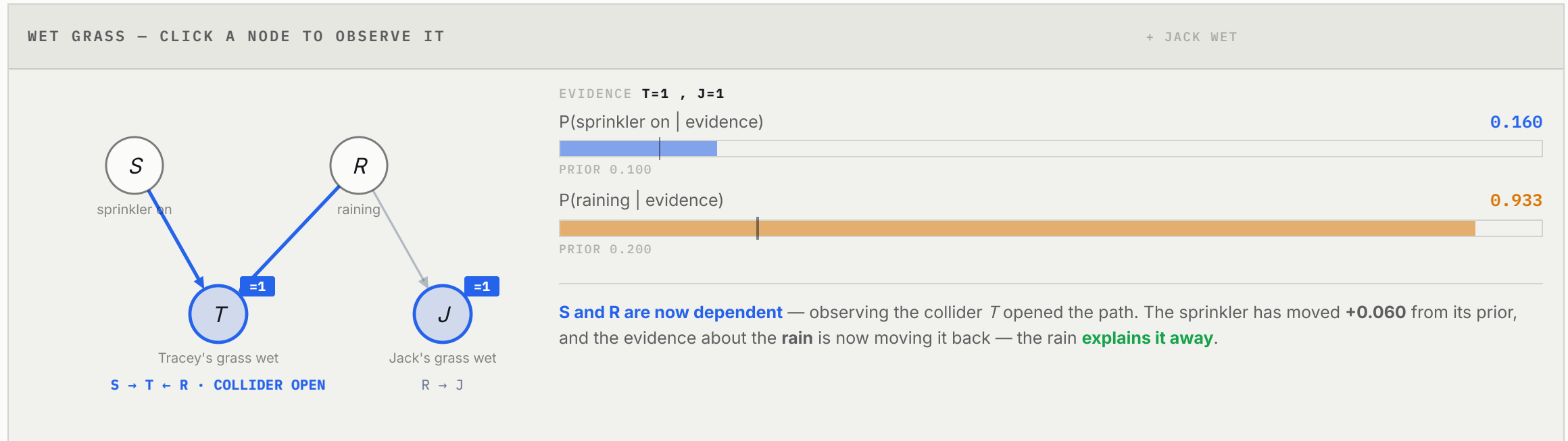
$$p(S=1 \mid T=1) = \frac{0.092}{0.272} = 0.338$$

$$p(S=1 \mid T=1, J=1) = \frac{0.0344}{0.2144} = 0.160$$

Tracey's wet grass more than triples the suspicion of the sprinkler. Then Jack's wet grass — **evidence for the rival cause** — halves it again.

Note that J is not on the collider path at all; it is a child of R . It moves the answer only because it moves R , and R competes with S to explain T . Note also that $p(S=1 \mid J=1) = 0.100$ exactly — with T unobserved, Jack's lawn says nothing whatever about Tracey's sprinkler.

Evidence, entered by hand



Click a node to observe it. The two blue edges are the collider path $S \rightarrow T \leftarrow R$; they light when observing T opens it. The readout is the exact posterior over the sprinkler, computed by enumerating all sixteen states. Observe T alone and the collider opens; **add J and the rain explains the sprinkler away**; observe R directly and the sprinkler snaps back to its prior, because with the rain known, wet grass says nothing.

The burglar alarm — the same move, sharper

Example 3.2

THE MODEL

B burgled · E earthquake · A alarm sounds · R radio reports a quake

$$B \rightarrow A \leftarrow E \rightarrow R$$

$p(B=1) = 0.01$, $p(E=1) = 10^{-6}$. The alarm is near-certain given either cause and almost never fires without one; the radio reports a quake exactly when there is one.

THE COLLAPSE

$$p(B=1) = 0.01$$

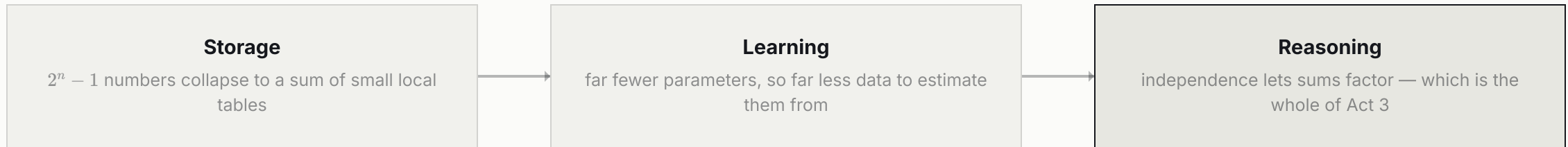
$$p(B=1 \mid A=1) = 0.99$$

$$p(B=1 \mid A=1, R=1) = 0.01$$

The alarm makes a burglary near-certain. One radio bulletin — which says nothing about burglars — sends it **straight back to the prior**.

R is not connected to B by any edge, and the two are marginally independent. They become strongly dependent the instant the alarm is heard. If you wanted one slide to justify learning d-separation properly, this is it.

What the missing edges buy



THE COMPLETE TEST

d-separation

Chaining the three structures across a whole graph gives **d-separation**: a purely graphical criterion that decides, for any three disjoint sets $\mathbf{X}$, $\mathbf{Y}$, $\mathbf{Z}$, whether the DAG entails $\mathbf{X} \perp \mathbf{Y} \mid \mathbf{Z}$. A path is blocked by an observed chain or fork node, and by an *unobserved* collider whose descendants are also unobserved. Block every path and the independence holds.

So the translation between graph and probability runs both ways and is exact: the edges you draw determine the independences, and the independences determine what the graph is allowed to cost.

— ACT 3

Reasoning with the network

Q3. A model you cannot query is decoration. Inference is where the factorisation earns its keep — and where it still, sometimes, fails.

The query, and three kinds of variable



Given evidence, what do we believe about something we cannot see? For the satellite: $P(B \mid d^1, c^1)$ — the chance of a battery failure, given a trajectory deviation and a communication loss.

ROLE	WHICH VARIABLES	IN THIS QUERY
query	what we want a distribution over	B
evidence	what we have observed, and fix	$D = d^1, C = c^1$
hidden	everything else — summed out	E, S

Every question a Bayesian network can answer has this shape. Diagnosis runs the arrows backwards (effects to causes); prediction runs them forwards; both are the same sum.

Exact inference — and why it hurts

Marginalise the joint over the hidden variables and normalise:

$$P(b^1 \mid d^1, c^1) \propto \sum_s \sum_e P(b^1)P(s)P(e \mid b^1, s)P(d^1 \mid e)P(c^1 \mid e)$$

This is correct, and it is a disaster. The number of terms is the number of joint assignments to the hidden variables — **exponential in how many there are**. The curse the factorisation was supposed to have removed comes back the moment we sum.

Storing the joint cheaply is not the same as **summing over it cheaply**.

Variable elimination — push each sum past what it cannot touch

$P(b^1)$ does not depend on s or e . $P(s)$ does not depend on e . So slide each sum inward until it meets a factor that actually mentions its variable:

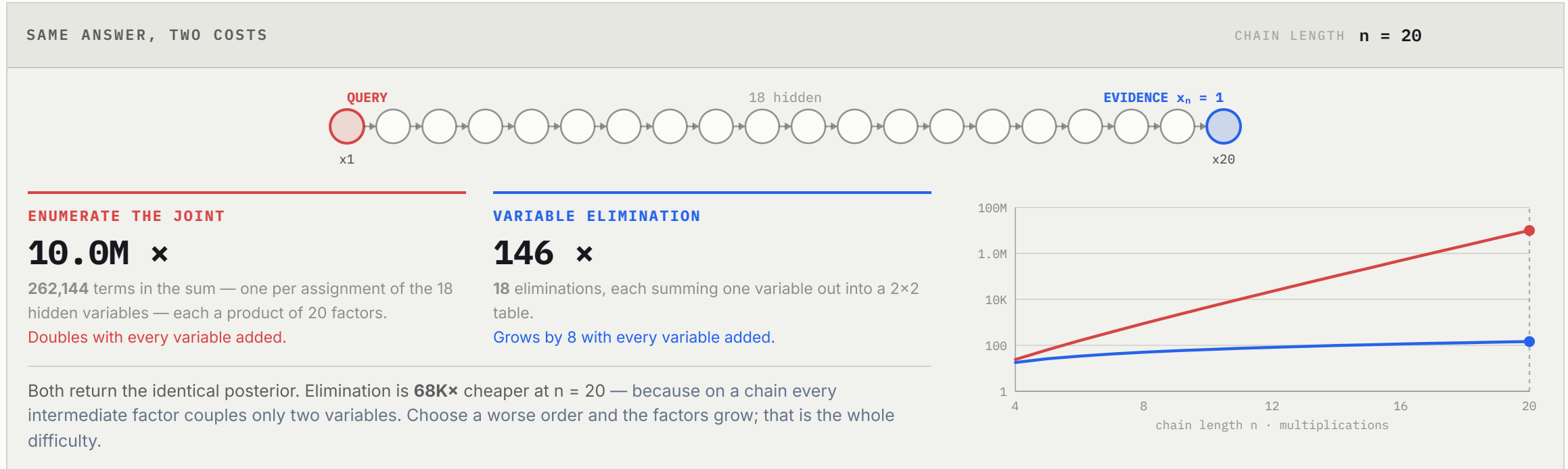
$$P(b^1 \mid d^1, c^1) \propto P(b^1) \sum_e P(d^1 \mid e) P(c^1 \mid e) \sum_s P(s) P(e \mid b^1, s)$$

Done systematically, this is an algorithm. Treat every conditional table as a **factor** over its variables; fix the evidence; then eliminate hidden variables one at a time, each elimination multiplying together the factors that mention it and summing it out into a new, smaller factor.

$$\underbrace{T_1(B) T_2(S) T_3(E, B, S) T_4(E) T_5(E)}_{\text{after fixing } d^1, c^1} \longrightarrow T_1(B) T_2(S) T_8(B, S) \longrightarrow T_1(B) T_9(B)$$

$T_8(B, S) = \sum_e T_3(e, B, S) T_6(e) T_7(e)$, then $T_9(B) = \sum_s T_2(s) T_8(B, s)$; normalise $T_1(B) T_9(B)$ and you have the answer. The cost is set by the largest intermediate factor, which depends on the **elimination order** — and finding the best order is itself NP-hard. So the ordering is a heuristic: often linear, **sometimes still exponential**.

What the ordering is worth



Both columns compute the identical number. On the left, enumeration: one term per assignment of the hidden variables, so the count doubles with every variable added. On the right, elimination: each hidden variable is summed out once into a small table, so the count grows by a constant. **At twenty variables it is 262 144 terms in one sum against 146 multiplications in total** — and both answers agree to the last digit.

When exact inference is hopeless — sample

DIRECT SAMPLING, WITH REJECTION

Walk the DAG in topological order, drawing each variable from $p(x_i \mid \text{pa}_i)$; keep only the runs that happen to match the evidence, and count.

Ten samples of the satellite network, three of which show d^1, c^1 , one of those with b^1 :

$$\hat{P}(b^1 \mid d^1, c^1) = 1/3$$

The flaw. If the evidence is unlikely, almost every sample is thrown away.

LIKELIHOOD WEIGHTING, AND ITS FLAW

Do not reject. Clamp each evidence variable to its observed value and carry a weight $w \leftarrow w \times P(x_i \mid \text{pa}_i)$ for the clamping.

The flaw is worse. Take $C \rightarrow D$ with $p(c^1) = 0.001$, $p(d^1 \mid c^1) = 0.999$, $p(d^1 \mid c^0) = 0.001$. The exact posterior is $p(c^1 \mid d^1) = 0.5$. But C is still drawn from its prior, so a sampler emits c^0 a thousand times before it ever sees c^1 — and reports **0 for an answer that is a half**.

The cure is to stop sampling variables independently: **Gibbs sampling** sweeps through the variables, redrawing each from its conditional given the current value of all the others, and — after a burn-in that is discarded — its samples come from the true posterior. Approximate inference is not a shortcut; it is a second set of failure modes, traded for the first.

The same graph, unrolled through time

Nothing so far said the variables were simultaneous. Index them by time and the identical machinery becomes a model of a system evolving.

$$P(S_1, \dots, S_T) = P(S_1) \prod_{t=2}^T P(S_t \mid S_{1:t-1}) \xrightarrow{\text{two assumptions}} P(S_1) \prod_{t=2}^T P(S_t \mid S_{t-1})$$

1 • THE MARKOV ASSUMPTION

$$P(S_t \mid S_1, \dots, S_{t-1}) = P(S_t \mid S_{t-1})$$

the future is conditionally independent of the past given the present,

$S_{t+1} \perp S_{1:t-1} \mid S_t$. Which is a *conditional independence* — a chain structure, read off a graph.

2 • THE STATIONARITY ASSUMPTION

$$P(S_{t+1}=s' \mid S_t=s) = P(S'=s' \mid S=s) \quad \forall t$$

one transition table, shared by every time step, instead of T of them.

Lecture 7 will open by *assuming* both of these. They are not assumptions about time; they are two missing-edge patterns in a Bayesian network over $S_1, \dots, S_T$, and everything Act 2 said about them still applies.

Filtering — Bayes' rule, once per time step

Hide the state and observe an emission — $X_t \rightarrow Y_t$ over a chain $X_{t-1} \rightarrow X_t$ — and you have a **hidden Markov model**. The standard query is *filtering*, the belief about now given everything seen so far:

$$P(x_t \mid y_{1:t}) \propto \underbrace{P(y_t \mid x_t)}_{\text{corrector}} \sum_{x_{t-1}} \underbrace{P(x_t \mid x_{t-1}) P(x_{t-1} \mid y_{1:t-1})}_{\text{predictor}}$$

Read it as Lecture 2's loop, run once per tick: yesterday's posterior is pushed through the dynamics to become today's **prior**; today's measurement is the **likelihood**; the product is today's posterior. Make the chain linear and the noise Gaussian and every term stays Gaussian — the recursion then carries only a mean and a covariance, and it has a name: the **Kalman filter**.

NOW ADD AN INPUT

and look what you have drawn

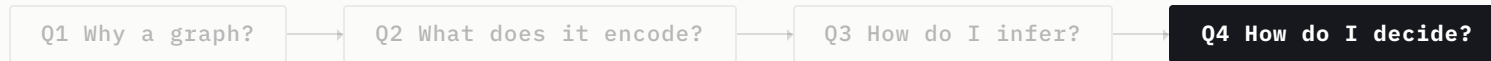
Let an action A_t steer the transition, $P(X_t \mid X_{t-1}, A_t)$, and the observation, $P(Y_t \mid X_t, A_t)$. That graph is a **POMDP**. Delete the emission row and observe the state directly, and it is a **Markov Decision Process**. Lecture 7 does not introduce a new object; it names this one.

— ACT 4

From belief to decision

Q4. Everything so far describes the world. Add two node types and the graph starts choosing.

Bayesian network + decision + utility



Bayesian network + decision node + utility node = decision network (influence diagram).

○ CHANCE

A random variable, exactly as before, with a conditional table given its parents. *Disease?*
Weather? *Demand?*

□ DECISION

A variable **we choose**. It has no distribution — that is the point. Its parents are what we get to see before choosing.

◇ UTILITY

A real-valued function of its parents. Several utility nodes are read as **added together**.

Three edge types come with them: a **conditional edge** into a chance node (the old kind), a **functional edge** into a utility node, and an **information edge** into a decision node — which does not carry probability at all, only the statement *this will be known when the choice is made*.

A worked example throughout: T *treat?*, D *disease?*, and O^1, O^2, O^3 *diagnostic test results*, with $U(T, D)$ giving 0 for a healthy untreated patient, -10 for a sick untreated one and -1 for treating either way. Those four numbers alone say: treat whenever $P(D=1) > 1/9$.

Utility — preference built the way belief was

BELIEF, FROM COMPARISONS

Given two statements you can say *I believe this more*, and if the comparison is complete and transitive it can be represented by a real-valued function with

$$P(A) > P(B) \iff A \succ B$$

That argument produced probability in Lecture 2.

PREFERENCE, FROM COMPARISONS

Given two *outcomes* you can say *I prefer this*. Add **continuity** — if $A \succ B \succ C$ there is a p with $[A:p; C:1-p] \sim B$ — and **monotonicity**, and the same argument produces a real-valued U .

Then the utility of a lottery is forced to be linear in the probabilities:

$$U([S_1:p_1; \dots; S_n:p_n]) = \sum_i p_i U(S_i)$$

Just as beliefs can be subjective, so can preferences — and U need not be money. The curvature of U against money *is* one's attitude to risk: concave is risk-averse, linear risk-neutral, convex risk-seeking. A **risk preference is a shape, not an extra ingredient**; expected utility already contains it.

Maximum expected utility

For an action a taken after seeing o , average the utility of the outcome over the belief the network gives you:

$$\text{EU}(a \mid o) = \sum_{s'} \underbrace{P(s' \mid o, a)}_{\text{the Bayesian network}} \underbrace{U(s')}_{\text{the utility node}}, \quad a^* = \arg \max_a \text{EU}(a \mid o)$$

The posterior comes from Act 3; the value comes from the utility node; **the arg max is the decision.**

Look at the letters. s' is an outcome state, a an action, $P(s' \mid o, a)$ a transition, $U(s')$ a reward. This is a one-step Bellman backup, written three lectures before there is a name for it. Nothing about it will change in Lecture 7 except that s' will have a successor.

Should I do a PhD?

Example 3.4 · a decision network with real numbers

THE NETWORK

E do PhD / no PhD is the **decision**; P win the prize? and I income: low / average / high are **chance**; U_C (cost of study) and U_B (benefit of income) are **utility**.

$$E \rightarrow P, \quad \{E, P\} \rightarrow I, \quad E \rightarrow U_C, \quad I \rightarrow U_B$$

$U_C(\text{do PhD}) = -50\,000$, $U_C(\text{no PhD}) = 0$; $U_B = 100\,000/200\,000/500\,000$ for low / average / high income; $p(\text{prize} \mid \text{PhD}) = 0.001$ against 10^{-7} without.

THE COMPUTATION

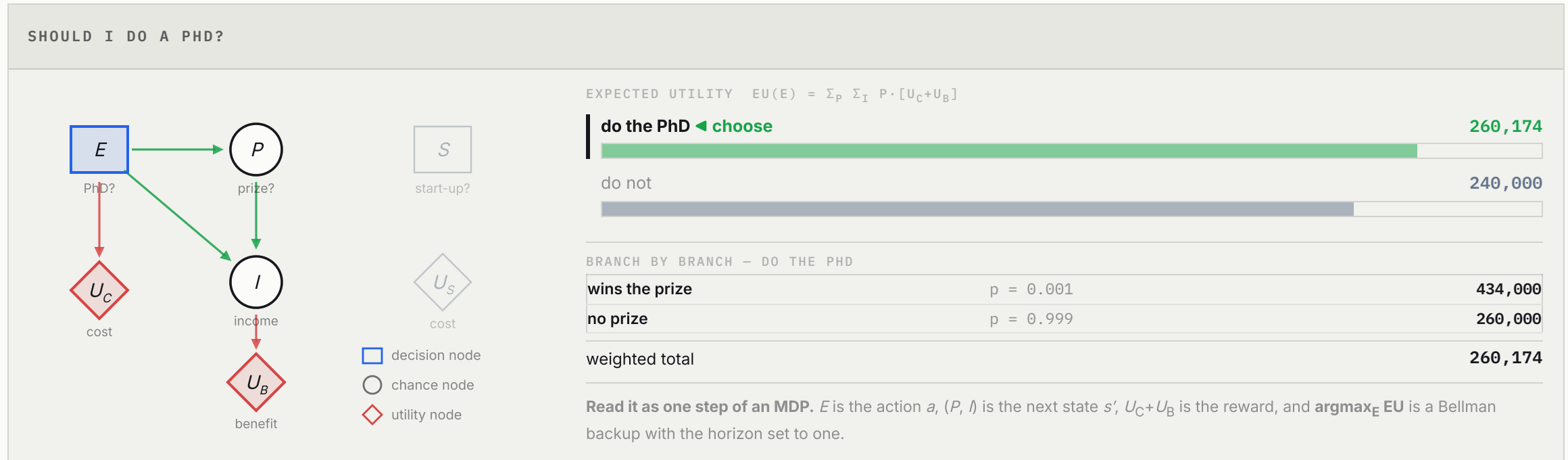
$$U(E) = \sum_{I,P} p(I \mid E, P) p(P \mid E) [U_C(E) + U_B(I)]$$

$$U(\text{do PhD}) = 260\,174$$

$$U(\text{no PhD}) = 240\,000$$

The doctorate wins by about 20 000 — and **almost none of that comes from the prize**. It comes from the income table: a PhD shifts the chance of high income from 0.2 to 0.4, which is worth more than the 50 000 it costs.

Pick a decision, watch the utility



The same network, live. Choose an action and the expected utility is computed by exactly the machinery of Acts 1–3: the network gives $p(I, P | E)$, the utility nodes give the value, the sum gives EU . Then switch on the second decision and [watch the recommendation reverse](#).

Add one option, and the answer flips

Example 3.5 · PhD and start-ups

WHAT CHANGES

A second decision S *found a start-up?* is added, with $U_S(\text{yes}) = -200\,000$, and an information edge $P \dashrightarrow S$ — the prize is known before the start-up is founded. Crucially, **income now hangs off S and P , not off E .**

$$U(E) = \sum_P \max_S \sum_I p(I \mid S, P) p(P \mid E) [U_C(E) + U_B(I) + U_S(S)]$$

WHAT HAPPENS

$$U(\text{do PhD}) = 190\,195$$

$$U(\text{no PhD}) = 240\,000$$

Do not do the PhD. And the start-up is never founded either — "no start-up" wins the inner **max** in every branch.

An option that is never exercised has **reversed the decision**, because the doctorate's only surviving route to income is a 0.001 chance at the prize, which does not repay 50 000.

No number in the utility tables changed. What changed is *which node the income depends on* — one edge. The graph, not the arithmetic, decided.

The value of information

If the decision can wait, it may pay to observe something first. Let $\text{EU}^*(o) = \max_a \text{EU}(a \mid o)$ be the value of deciding well given what is known. Then observing a new variable is worth

$$\text{VOI}(O^{\text{new}} \mid o) = \left(\sum_{o^{\text{new}}} P(o^{\text{new}} \mid o) \text{EU}^*(o^{\text{new}}, o) \right) - \text{EU}^*(o)$$

the expected value of the *better decision* the observation lets you make, minus what you would have got anyway. It is never negative — information cannot hurt a rational agent — and it must be weighed against the **cost of the observation**, which the formula does not include.

WHERE YOU WILL MEET THIS AGAIN

Lecture 4

"How much is it worth to look here?" is precisely the question an **acquisition function** answers in Bayesian optimisation. Expected improvement and the knowledge gradient are value of information, computed against a Gaussian process instead of a discrete network. Act 4 has already written the formula.

Why this is the hinge of the whole course

A decision network is the first object in this course to hold all three ingredients of sequential decision making at once:

belief (chance nodes) + **action** (decision node) + **value** (utility node)

Now allow several decisions. An influence diagram fixes a partial ordering $\mathcal{X}_0 \prec D_1 \prec \mathcal{X}_1 \prec D_2 \prec \dots \prec D_n \prec \mathcal{X}_n$ — what is revealed between choices — and the optimal first decision is

$$U(d_1 \mid x_1) = \sum_{x_2} \max_{d_2} \sum_{x_3} \max_{d_3} \dots \prod_t p(x_{t+1} \mid x_t, d_t) \sum_t u(x_t), \quad d_1^* = \arg \max_{d_1} U(d_1 \mid x_1)$$

That alternating $\sum \max \sum \max$ is [the Bellman equation](#), four lectures early.

$p(x_{t+1} \mid x_t, d_t)$ is the transition $P(s' \mid s, a)$; the additive $\sum_t u(x_t)$ is the return; the inner $\max$ is the optimal policy assumed for the future. Lecture 7 will give this a name, add a discount, and solve it by dynamic programming. Lecture 8 will delete p and learn it from samples. [Everything in Part IV is this one slide, extended through time and stripped of its model.](#)

— CLOSING

Closing

Belief structured, and action attached. Two roads lead out of here, and they are the rest of the course.

Where we are — the graph, and the seed

MODEL - BASED		DATA - DRIVEN
Static, single	optimisation (<i>Lec 1 ✓</i>)	Bayesian statistics (<i>Lec 2 ✓</i>) → Bayesian networks (<i>Lec 3 ✓</i>) → Lec 4

We can now represent belief over a whole system as a graph, read its independences off the missing edges, answer questions within it by elimination or by sampling, and — newly — attach a decision and a value to it.

This lecture hands on [structured belief, plus decision and utility nodes — the influence diagram, which is a one-stage MDP](#).

LECTURE 4 — BELIEF THAT ACTS

Put belief to work on an *unknown function*. A Gaussian process is a Bayesian network over a continuum of variables; a Bayesian optimiser chooses where to look next by [value of information](#), which Act 4 already wrote down.

LECTURE 7 — BELIEF THROUGH TIME

Unroll the decision network. One decision becomes a sequence, the utility node becomes a reward per step, and $\arg \max_a EU$ becomes [the Bellman optimality operator](#).

A joint distribution over a system is unmanageable
— until conditional independence draws it as a
graph.

Small local tables in place of one exponential one; sums that factor because the edges are missing; and then a decision node and a value node, which together are, in miniature, every decision problem still to come.

Questions?

Remember the decision network. It is the static, one-shot ancestor of the MDP, of dynamic programming, and of the reinforcement learner — belief, action and value, before time enters the picture.

— APPENDIX

Backup slides

The complete arguments, kept out of the narrative.

Backup 1 — variable elimination, step by step

Query $P(B \mid d^1, c^1)$ on $p(B, S, E, D, C) = p(B)p(S)p(E \mid B, S)p(D \mid E)p(C \mid E)$. Write each conditional table as a factor over its variables:

$$T_1(B) \ T_2(S) \ T_3(E, B, S) \ T_4(D, E) \ T_5(C, E)$$

1. **Insert the evidence.** Fix $D = d^1$ and $C = c^1$, which reduces T_4 and T_5 to factors over E alone: $T_6(E) = T_4(d^1, E)$, $T_7(E) = T_5(c^1, E)$.
2. **Eliminate E .** Multiply every factor mentioning E and sum it out: $T_8(B, S) = \sum_e T_3(e, B, S) T_6(e) T_7(e)$.
3. **Eliminate S .** $T_9(B) = \sum_s T_2(s) T_8(B, s)$.
4. **Combine and normalise.** $P(B \mid d^1, c^1) \propto T_1(B) T_9(B)$.

Cost. Each elimination creates an intermediate factor whose size is 2^k where k is the number of variables it couples — the *induced width* of the chosen ordering. A good ordering keeps every factor small; a bad one can build a factor over most of the network. Finding the optimal ordering is NP-hard, so heuristics (min-fill, min-degree) are used. This is the practical ceiling of exact inference, and the reason sampling exists.

Backup 2 — d-separation, the complete rule

A path between $\mathbf{X}$ and $\mathbf{Y}$ (ignoring edge directions) is **blocked** by an observed set $\mathbf{Z}$ if it contains a node m such that either

- m is a **chain** ($\rightarrow m \rightarrow$) or a **fork** ($\leftarrow m \rightarrow$) on the path **and** $m \in \mathbf{Z}$; or
- m is a **collider** ($\rightarrow m \leftarrow$) on the path **and** neither m nor any descendant of m is in $\mathbf{Z}$.

If **every** path is blocked, $\mathbf{Z}$ d-separates $\mathbf{X}$ from $\mathbf{Y}$, and the DAG entails $\mathbf{X} \perp \mathbf{Y} \mid \mathbf{Z}$.

WHAT IT GUARANTEES

D-separation is **sound**: if it says independent, every distribution factorising over the DAG has that independence. So a graphical check licenses an algebraic simplification, with no arithmetic.

WHAT IT DOES NOT

The converse fails in one direction: $\mathbf{X}$ and $\mathbf{Y}$ may happen to be independent in a particular distribution without being d-separated — an accident of the numbers rather than the structure. The graph states what *must* hold, not everything that does.

The "or any descendant" clause is why observing J in the wet-grass example matters but observing it *without* T does not: J is a descendant of R , not of the collider T , so on its own it opens nothing.

Backup 3 — approximate inference, and where each method breaks

METHOD	HOW IT WORKS	WHERE IT BREAKS
Direct (rejection) sampling	draw whole assignments in topological order; discard those disagreeing with the evidence	rare evidence throws away almost every sample
Likelihood weighting	clamp the evidence, weight each sample by $\prod P(x_i \mid \text{pa}_i)$ over the clamped nodes	the <i>non</i> -evidence variables are still drawn from their priors, so a rare cause is never proposed
Gibbs sampling	redraw each variable from its conditional given all the others; discard a burn-in	correct in the limit, but can mix arbitrarily slowly

The likelihood-weighting counter-example. Take $C \rightarrow D$ with $p(c^1) = 0.001$, $p(d^1 \mid c^1) = 0.999$, $p(d^1 \mid c^0) = 0.001$, so that exactly $p(c^1 \mid d^1) = \frac{0.999 \times 0.001}{0.999 \times 0.001 + 0.001 \times 0.999} = 0.5$. But C is sampled from its prior, so a run of a thousand draws is $c^0, c^0, \dots$ and the weighted estimate is 0. The estimator is unbiased and, at any practical sample size, useless — "asymptotically correct" is a statement about a limit you may never reach.

Backup 4 — from maximum expected utility to Bellman

One decision. Chance variables X with a Bayesian network, a decision D , a utility $U(D, X)$. Given evidence e ,

$$\text{EU}(d \mid e) = \sum_x p(x \mid d, e) U(d, x), \quad d^* = \arg \max_d \text{EU}(d \mid e), \quad \text{MEU}(e) = \max_d \text{EU}(d \mid e)$$

Many decisions. With the partial ordering $\mathcal{X}_0 \prec D_1 \prec \dots \prec D_n \prec \mathcal{X}_n$, the value of the first decision is

$$U(d_1 \mid x_0) = \sum_{\mathcal{X}_1} \max_{D_2} \dots \sum_{\mathcal{X}_{n-1}} \max_{D_n} \sum_{\mathcal{X}_n} \prod_{i \in \mathcal{L}} p(x_i \mid \text{pa}(x_i)) \sum_{j \in \mathcal{T}} U_j(\text{pa}(u_j))$$

with $\mathcal{L}$ the chance variables and $\mathcal{T}$ the utility variables.

The translation. Let each decision move a state, $p(x_{t+1} \mid x_t, d_t)$; let utility accumulate as a per-step reward, $\sum_t u(x_t)$. Then MEU becomes the value function V^* , the alternating $\sum \max$ becomes the dynamic-programming recursion, and $\arg \max_d \text{EU}$ becomes the Bellman optimality operator. Lecture 7 adds a discount factor γ and an infinite horizon; Lecture 8 removes p and estimates the expectation from samples. The object itself was built here.